華南師範大學

本科学生实验（实践）报告

院　　系：计 算 机 学 院

实验课程：编译原理

实验项目：正则表达式转换为 DFA 图

指导老师：黄煜廉

开课时间：2025 ～ 2026 年度第 2 学期

专　　业：计算机科学与技术

班　　级：24 级 计算机科学与技术 2 班

学　　生：廖艺琳

学　　号：20230739041

华南师范大学教务处

华南师范大学实验报告

学生姓名_____学　　号_____

专　　业_____年级、班级_____

课程名称_____实验项目_____

实验类型　　□验证 □设计 □综合　实验时间_____年____月____日

实验指导老师_____实验评分_____

一、实验内容

设计一个应用软件，以实现将正则表达式-->NFA--->DFA-->DFA 最小化-->词法分析程序(选做内容)

二、必做实验内容及要求：

（1）正则表达式应该可以支持命名操作，运算符号有： 转义符号(\)、连接、 选择(|)、 闭包(*)、 正闭包（+)、 范围（[ ]）、 可选（?)、括号 ( )

（2）要提供一个源程序编辑界面，让用户输入一行（一个）或多行（多个）正则表达式（可保存、打开正则表达式文件）

（3）需要提供窗口以便用户可以查看转换得到的 NFA（用状态转换表呈现即可）

（4）需要提供窗口以便用户可以查看转换得到的 DFA（用状态转换表呈现即可）

（5）需要提供窗口以便用户可以查看转换得到的最小化 DFA（用状态转换表呈现即可）

（6）要求应用程序为窗口式图形化界面

三、选做实验内容及要求：

（1）将最小化得到 DFA 图转换得到的词法分析源程序（该分析程序需要用 C/C++语言描述，而且只能采用讲稿中的转换方法一或转换方法二来生成源程序。)，系统需提供窗口以便用户可以查看转换得到的词法分析源程序

（2）书写完善的实验报告书，书写格式请阅读文件：实验报告书（含软件文档） 版头及正文格式.doc

二、实验目的

1. 掌握正则表达式、NFA、DFA 的基本原理与相互转换算法。
2. 实现 Thompson 构造、子集构造、DFA 最小化三大核心自动机算法。
3. 基于自动机实现简易词法分析引擎，理解词法分析的底层实现逻辑。
4. 掌握 C++ 容器、栈、映射、状态管理等工程化编码能力。
5. 能够生成可独立运行的词法分析代码，加深对编译前端的理解。

三、实验文档：

1.系统总体结构

系统采用 C++ + Qt 6 开发，整体分为界面层、自动机层、词法层三层架构：

1. 界面层（MainWindow）：提供正则输入、文件加载、NFA/DFA 表格展示、生成代码预览等交互功能，响应用户操作并调用底层引擎。
2. 自动机层（Automata::Engine）：核心算法模块，实现正则预处理、中缀转后缀、NFA 构造、NFA 转 DFA、DFA 最小化、多 NFA 合并。
3. 词法层（Lexer::LexerEngine）：基于最小 DFA 实现输入串模拟匹配，生成可独立编译运行的 C++ 词法分析代码。

模块交互流程

1. 用户输入正则表达式 / 加载规则文件 → 界面层传递给自动机引擎。
2. 自动机引擎完成正则→NFA→DFA→最小 DFA 的转换。
3. 界面层将 NFA、DFA、最小 DFA 状态填充到表格可视化。
4. 词法引擎根据最小 DFA 生成 C++ 代码并展示。

2.数据结构设计

系统设计了精简高效的数据结构来管理解析过程：

1. NFA 相关结构

```
// NFA 状态结构
struct NFAState {
```

```
    int id;
    bool isAccept;
    int acceptTokenId;
    QMap<int, QVector<int>> transitions;
    NFAState(int id) : id(id), isAccept(false), acceptTokenId(-1) {}
};
// NFA 图结构
struct NFAGraph {
    int startState;
    int endState;
    QVector<NFAState*> states;
    NFAGraph() : startState(-1), endState(-1) {}
};
```

2.DFA 相关结构

```
// DFA 状态结构
struct DFAState {
    int id;
    bool isAccept;
    int acceptTokenId;
    QSet<int> nfaStates;
    QMap<int, int> transitions;
    DFAState(int id) : id(id), isAccept(false), acceptTokenId(-1) {}
};
// DFA 图结构
struct DFAGraph {
    int startState;
    QVector<DFAState*> states;
    QSet<int> alphabet;
    DFAGraph() : startState(-1) {}
```

};

3.关键算法设计方案

3.1 正则表达式预处理+中缀转后缀

移除空白字符，处理转义字符、字符集[]、用户别名。

自动插入连接符.，将 ab 转为 a.b，统一运算规则。

使用运算符栈，按优先级*+/+.>|?处理。

遇到左括号入栈，右括号则出栈直到左括号，输出合法后缀串。

```
// 预处理正则表达式：处理转义、字符类、别名，并插入显式的连接符号 '.'
QString Engine::preprocessRegex(QString regex, const QSet<QString>& aliases) {
    QStringList tokens;

    for (int i = 0; i < regex.length(); ++i) {
        if (regex[i].isSpace()) continue;

        // 处理转义字符
        if (regex[i] == '\\' && i + 1 < regex.length()) {
            tokens << regex.mid(i, 2);
            i++;
            continue;
        }

        // 处理字符类 [A-Za-z] 等
        if (regex[i] == '[') {
            int end = regex.indexOf(']', i);
            if (end != -1) {
                tokens << regex.mid(i, end - i + 1);
                i = end;
                continue;
            }
        }

        // 处理别名（如 letter, digit）
        if (regex[i].isLetter() || regex[i] == '_') {
            QString id;
            while (i < regex.length() && (regex[i].isLetterOrNumber() || regex[i] == '_')) {
                id += regex[i++];
            }
            i--;
            tokens << id;
            continue;
        }

        tokens << QString(regex[i]);
    }

    // 自动插入隐式的连接符 '.' (例如 "ab" 变为 "a.b")
    QString processed;
    for (int i = 0; i < tokens.size(); ++i) {
        QString t1 = tokens[i];
        processed += t1;

        if (i + 1 < tokens.size()) {
            QString t2 = tokens[i+1];

            bool t1CanConcat = (t1 != "(" && t1 != "|" && t1 != "\\");
            bool t2NeedsConcat = (t2 != ")" && t2 != "|" && t2 != "*" && t2 != "+" && t2 != "?" && t2 != ".");

            if (t1 == "*" || t1 == "+" || t1 == "?" || t1 == ")") t1CanConcat = true;

            if (t1CanConcat && t2NeedsConcat) {
                processed += ".";
            }
        }
    }
    return processed;
}
```

```cpp
// 中缀转后缀
QString Engine::infixToPostfix(QString infix, const QSet<QString>& aliases) {
    QString postfix;
    QStack<QString> stack;

    for (int i = 0; i < infix.length(); ++i) {
        QString t;
        QChar c = infix[i];

        if (c == '\\' && i + 1 < infix.length()) {
            t = infix.mid(i, 2);
            i++;
            postfix += t + " ";
            continue;
        }

        if (c == '(') {
            stack.push("(");
            continue;
        }

        if (c == ')') {
            while (!stack.isEmpty() && stack.top() != "(") {
                postfix += stack.pop() + " ";
            }
            if (!stack.isEmpty()) stack.pop();
            continue;
        }

        if (c == '|' || c == '*' || c == '+' || c == '?' || c == '.') {
            while (!stack.isEmpty() && getPrecedence(stack.top()[0]) >= getPrecedence(c)) {
                postfix += stack.pop() + " ";
            }
            stack.push(QString(c));
            continue;
        }

        // 处理别名、字符类或普通字符
        if (c.isLetter() || c == '_') {
            while (i < infix.length() && (infix[i].isLetterOrNumber() || infix[i] == '_')) {
                t += infix[i++];
            }
            i--;
        } else if (c == '[') {
            int end = infix.indexOf(']', i);
            if (end != -1) {
                t = infix.mid(i, end - i + 1);
                i = end;
            } else t = c;
        } else {
            t = c;
        }
        postfix += t + " ";
    }

    while (!stack.isEmpty()) {
        postfix += stack.pop() + " ";
    }
    return postfix.trimmed();
}
```

3.2 Thompson 构造法（正则→NFA）

1. 单字符 NFA：两个状态 + 一条带字符转移边。
2. 连接（ab）：前一个 NFA 的终态用 ε 转移指向后一个 NFA 的初态。
3. 选择（a|b）：新建起点 ε 指向两个分支起点，两个分支终态 ε 指向共同终点。
4. 闭包（a）：允许重复 0 次或多次，增加绕回与跳过分支。
5. 正闭包（a+）/可选（a?）：在闭包基础上做简单变形

```cpp
// Thompson 构造法：创建一个接受单个字符的 NFA
NFAGraph* Engine::createBasic(int c) {
    NFAGraph* nfa = new NFAGraph();
    NFAState* s1 = new NFAState(newStateId());
    NFAState* s2 = new NFAState(newStateId());
    s1->transitions[c].push_back(s2->id);
    nfa->states.push_back(s1);
    nfa->states.push_back(s2);
    nfa->startState = s1->id;
    nfa->endState = s2->id;
    s2->isAccept = true;
    return nfa;
}

// Thompson 构造法：并运算 (a|b)
NFAGraph* Engine::createUnion(NFAGraph* left, NFAGraph* right) {
    NFAGraph* nfa = new NFAGraph();
    NFAState* sStart = new NFAState(newStateId());
    NFAState* sEnd = new NFAState(newStateId());

    sStart->transitions[EPSILON].push_back(left->startState);
    sStart->transitions[EPSILON].push_back(right->startState);

    for (auto s : left->states) if (s->id == left->endState) { s->isAccept = false; s->transitions[EPSILON].push_back(sEnd->id); }
    for (auto s : right->states) if (s->id == right->endState) { s->isAccept = false; s->transitions[EPSILON].push_back(sEnd->id); }

    nfa->states.push_back(sStart);
    nfa->states.append(left->states);
    nfa->states.append(right->states);
    nfa->states.push_back(sEnd);

    nfa->startState = sStart->id;
    nfa->endState = sEnd->id;
    sEnd->isAccept = true;

    delete left; delete right;
    return nfa;
}

// Thompson 构造法：连接运算 (ab)
NFAGraph* Engine::createConcat(NFAGraph* first, NFAGraph* second) {
    for (auto s : first->states) {
        if (s->id == first->endState) {
            s->isAccept = false;
            s->transitions[EPSILON].push_back(second->startState);
        }
    }
    first->states.append(second->states);
    first->endState = second->endState;
    delete second;
    return first;
}

// Thompson 构造法：闭包运算 (a*)
NFAGraph* Engine::createStar(NFAGraph* inner) {
    NFAGraph* nfa = new NFAGraph();
    NFAState* sStart = new NFAState(newStateId());
    NFAState* sEnd = new NFAState(newStateId());

    sStart->transitions[EPSILON].push_back(inner->startState);
    sStart->transitions[EPSILON].push_back(sEnd->id);
```

```cpp
// Thompson 构造法：闭包运算 (a*)
NFAGraph* Engine::createStar(NFAGraph* inner) {
    NFAGraph* nfa = new NFAGraph();
    NFAState* sStart = new NFAState(newStateId());
    NFAState* sEnd = new NFAState(newStateId());

    sStart->transitions[EPSILON].push_back(inner->startState);
    sStart->transitions[EPSILON].push_back(sEnd->id);

    for (auto s : inner->states) {
        if (s->id == inner->endState) {
            s->isAccept = false;
            s->transitions[EPSILON].push_back(inner->startState);
            s->transitions[EPSILON].push_back(sEnd->id);
        }
    }

    nfa->states.push_back(sStart);
    nfa->states.append(inner->states);
    nfa->states.push_back(sEnd);
    nfa->startState = sStart->id;
    nfa->endState = sEnd->id;
    sEnd->isAccept = true;

    delete inner;
    return nfa;
}

// Thompson 构造法：正闭包 (a+)
NFAGraph* Engine::createPlus(NFAGraph* inner) {
    NFAGraph* nfa = new NFAGraph();
    NFAState* sStart = new NFAState(newStateId());
    NFAState* sEnd = new NFAState(newStateId());

    sStart->transitions[EPSILON].push_back(inner->startState);

    for (auto s : inner->states) {
        if (s->id == inner->endState) {
            s->isAccept = false;
            s->transitions[EPSILON].push_back(inner->startState);
            s->transitions[EPSILON].push_back(sEnd->id);
        }
    }

    nfa->states.push_back(sStart);
    nfa->states.append(inner->states);
    nfa->states.push_back(sEnd);
    nfa->startState = sStart->id;
    nfa->endState = sEnd->id;
    sEnd->isAccept = true;

    delete inner;
    return nfa;
}
```

3.3 子集构造法（NFA→DFA）

1. ε- 闭包

   从一个状态 / 状态集出发，**经过任意条 ε 转移能到达的所有状态**。

2. Move 函数

   给定状态集与字符，给出一步转移能到达的状态集合。

3. 子集构造流程

   从起始状态的 ε- 闭包开始，作为 DFA 初始状态。

   对每个新状态，遍历所有字符，计算 Move + 闭包得到新状态。

   直到没有新状态产生，DFA 构造完成。

```
// 计算 ε-闭包
QSet<int> Engine::epsilonClosure(NFAGraph* nfa, QSet<int> states) {
    QSet<int> closure = states;
    QStack<int> stack;
    for (int s : states) stack.push(s);

    QMap<int, NFAState*> stateMap;
    for (auto s : nfa->states) stateMap[s->id] = s;

    while (!stack.isEmpty()) {
        int u = stack.pop();
        if (stateMap.contains(u)) {
            for (int v : stateMap[u]->transitions[EPSILON]) {
                if (!closure.contains(v)) {
                    closure.insert(v);
                    stack.push(v);
                }
            }
        }
    }
    return closure;
}

// 子集构造法：计算从给定状态集合出发，通过某个符号能到达的所有 NFA 状态
QSet<int> Engine::move(NFAGraph* nfa, const QSet<int>& states, int symbol) {
    QSet<int> result;
    QMap<int, NFAState*> stateMap;
    for (auto s : nfa->states) stateMap[s->id] = s;

    for (int s : states) {
        if (stateMap.contains(s) && stateMap[s]->transitions.contains(symbol)) {
            for (int target : stateMap[s]->transitions[symbol]) {
                result.insert(target);
            }
        }
    }
    return result;
}
```

```cpp
// NFA 转 DFA (子集构造法)
DFAGraph* Engine::nfaToDFA(NFAGraph* nfa) {
    if (!nfa) return nullptr;
    DFAGraph* dfa = new DFAGraph();
    QSet<int> alphabet;
    for (auto s : nfa->states) {
        for (auto it = s->transitions.begin(); it != s->transitions.end(); ++it) {
            if (it.key() != EPSILON) alphabet.insert(it.key());
        }
    }
    dfa->alphabet = alphabet;

    QVector<QSet<int>> dstates;
    QSet<int> startClosure = epsilonClosure(nfa, {nfa->startState});
    dstates.push_back(startClosure);

    DFAState* dStart = new DFAState(1);
    dStart->nfaStates = startClosure;
    for (int nsId : startClosure) {
        for (auto ns : nfa->states) {
            if (ns->id == nsId && ns->isAccept) {
                dStart->isAccept = true;
                if (dStart->acceptTokenId == -1 || ns->acceptTokenId < dStart->acceptTokenId) {
                    dStart->acceptTokenId = ns->acceptTokenId;
                }
            }
        }
    }
    dfa->states.push_back(dStart);
    dfa->startState = 1;

    int i = 0;
    while (i < dstates.size()) {
        QSet<int> T = dstates[i];
        for (int symbol : alphabet) {
            QSet<int> U = epsilonClosure(nfa, move(nfa, T, symbol));
            if (U.isEmpty()) continue;

            int targetIdx = -1;
            for (int k = 0; k < dstates.size(); ++k) {
                if (dstates[k] == U) { targetIdx = k + 1; break; }
            }

            if (targetIdx == -1) {
                targetIdx = dstates.size() + 1;
                dstates.push_back(U);
                DFAState* dNew = new DFAState(targetIdx);
                dNew->nfaStates = U;
                for (int nsId : U) {
                    for (auto ns : nfa->states) {
                        if (ns->id == nsId && ns->isAccept) {
                            dNew->isAccept = true;
                            if (dNew->acceptTokenId == -1 || ns->acceptTokenId < dNew->acceptTokenId) {
                                dNew->acceptTokenId = ns->acceptTokenId;
                            }
                        }
                    }
                }
                dfa->states.push_back(dNew);
            }
            dfa->states[i]->transitions[symbol] = targetIdx;
        }
        i++;
    }
    return dfa;
}
```

3.5 DFA 最小化（分组划分法）

初始分组：按是否接受态 + 接受 TokenID 划分。

迭代分裂：同一组内状态转移目标不在同一组则分裂。

重构最小 DFA，减少状态数，提升匹配效率。

```cpp
// 最小化 DFA (分组划分法)
DFAGraph* Engine::minimizeDFA(DFAGraph* dfa) {
    if (!dfa || dfa->states.isEmpty()) return dfa;

    // 初始划分：根据是否为接受状态以及接受的 Token ID 进行分组
    QMap<int, QSet<int>> initialGroups;
    for (auto s : dfa->states) {
        int key = s->isAccept ? s->acceptTokenId : -1;
        initialGroups[key].insert(s->id);
    }
    QVector<QSet<int>> partitions;
    for (auto group : initialGroups.values()) partitions.push_back(group);    ⚠allocating ar

    // 迭代划分直到状态不再变化
    bool changed = true;
    while (changed) {
        changed = false;
        QVector<QSet<int>> newPartitions;
        for (const auto& group : partitions) {
            if (group.size() <= 1) {
                newPartitions.push_back(group);
                continue;
            }

            QMap<int, QSet<int>> splitMap;
            for (int sId : group) {
                DFAState* s = nullptr;
                for (auto state : dfa->states) if (state->id == sId) { s = state; break; }

                // 根据在当前划分下的转换目标生成特征指纹
                QString signature;
                for (int symbol : dfa->alphabet) {
                    int target = s->transitions.value(symbol, -1);
                    int partIdx = -1;
                    if (target != -1) {
                        for (int k = 0; k < partitions.size(); ++k) {
                            if (partitions[k].contains(target)) { partIdx = k; break; }
                        }
                    }
                    signature += QString::number(partIdx) + ",";
                }
                int sigHash = qHash(signature);
                splitMap[sigHash].insert(sId);
            }

            if (splitMap.size() > 1) changed = true;
            for (auto it = splitMap.begin(); it != splitMap.end(); ++it) {
                newPartitions.push_back(it.value());
            }
        }
        partitions = newPartitions;
    }
```

```
    // 根据划分结果重新构建 DFA
    DFAGraph* minDfa = new DFAGraph();
    minDfa->alphabet = dfa->alphabet;
    QMap<int, int> stateToPart;
    for (int i = 0; i < partitions.size(); ++i) {
        for (int sId : partitions[i]) stateToPart[sId] = i + 1;
    }

    for (int i = 0; i < partitions.size(); ++i) {
        DFAState* dNew = new DFAState(i + 1);
        int repId = *partitions[i].begin();
        DFAState* repState = nullptr;
        for (auto s : dfa->states) if (s->id == repId) { repState = s; break; }

        dNew->isAccept = repState->isAccept;                    ⚠ Acc
        dNew->acceptTokenId = repState->acceptTokenId;
        for (int symbol : dfa->alphabet) {
            if (repState->transitions.contains(symbol)) {
                dNew->transitions[symbol] = stateToPart[repState->transitions[symbol]];
            }
        }
        minDfa->states.push_back(dNew);
        if (partitions[i].contains(dfa->startState)) minDfa->startState = i + 1;
    }

    return minDfa;
}

}
```

3.6 词法分析代码生成

1. DFA 模拟

从起始状态开始，按输入字符一步步转移。

记录最后一次到达接受状态的位置，保证最长匹配。

无法转移时停止，返回匹配长度。

2. 代码生成

把最小 DFA 直接翻译成可独立运行的 C++ 代码。

```cpp
int LexerEngine::simulateDFA(Automata::DFAGraph* dfa, const QString& input, int start, const QMap<int, QString>& idToAlias, Automata::Engine& engine) {
    if (!dfa) return 0;

    int currentStateId = dfa->startState;
    int lastAcceptLen = 0;
    int currentLen = 0;

    auto findState = [&](int id) -> Automata::DFAState* {
        for (auto s : dfa->states) if (s->id == id) return s;
        return nullptr;
    };

    // 遍历输入字符串进行DFA状态转移
    while (start + currentLen < input.length()) {
        int c = input[start + currentLen].unicode();

        Automata::DFAState* s = findState(currentStateId);
        if (!s) break;

        bool found = false;
        if (s->transitions.contains(c)) {
            currentStateId = s->transitions[c];
            found = true;
        } else {
            // 检查别名转换
            for (auto it = s->transitions.begin(); it != s->transitions.end(); ++it) {
                int sym = it.key();
                if (idToAlias.contains(sym)) {
                    QString pattern = engine.getAliasPattern(idToAlias[sym]);
                    if (pattern == "[A-Za-z]" && ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'))) {
                        currentStateId = it.value();
                        found = true;
                        break;
                    } else if (pattern == "[0-9]" && (c >= '0' && c <= '9')) {
                        currentStateId = it.value();
                        found = true;
                        break;
                    }
                }
            }
        }

        if (!found) break;
        currentLen++;

        Automata::DFAState* nextS = findState(currentStateId);
        if (nextS && nextS->isAccept) {
            lastAcceptLen = currentLen;
        }
    }

    return lastAcceptLen;
}
```

```
// 词法分析核心函数
code += "void GetToken() {\n";
code += "    int state = " + QString::number(dfa->startState) + ";\n";
code += "    int lastAcceptId = -1;\n";
code += "    int lastAcceptPos = -1;\n";
code += "    int startPos = pos - 1;\n\n";

code += "    while (true) {\n";
code += "        switch (state) {\n";

for (auto s : dfa->states) {
    code += QString("            case %1:\n").arg(s->id);
    if (s->isAccept) {
        code += "                lastAcceptId = " + QString::number(s->acceptTokenId) + ";\n";
        code += "                lastAcceptPos = pos;\n";
    }

    if (s->transitions.isEmpty()) {
        code += "                goto end_loop;\n";
        continue;
    }

    QMap<int, QList<int>> targets;
    for (auto it = s->transitions.begin(); it != s->transitions.end(); ++it) {
        targets[it.value()].append(it.key());
    }

    bool first = true;
    for (auto it = targets.begin(); it != targets.end(); ++it) {
        QList<int> chars = it.value();
        QStringList conditions;
        for (int c : chars) {
            if (idToAlias.contains(c)) {
                QString pattern = engine.getAliasPattern(idToAlias[c]);
                if (pattern == "[A-Za-z]") conditions << "Judgeletter(ch)";
                else if (pattern == "[0-9]") conditions << "Judgedigit(ch)";
                else conditions << QString("(ch == %1)").arg(c);
            } else {
                conditions << QString("(ch == %1)").arg(c);
            }
        }
        code += QString("                %1 (%2) state = %3;\n")
                    .arg(first ? "if" : "else if")
                    .arg(conditions.join(" || "))
                    .arg(it.key());
        first = false;
    }
    code += "                else goto end_loop;\n";
    code += "                break;\n";
}

code += "        }\n";
code += "        ch = GetNext();\n";
code += "        if (ch == 0) break;\n";
code += "    }\n\n";

code += "end_loop:\n";
code += "    if (lastAcceptId != -1) {\n";
code += "        // 回溯到最后一个接受位置\n";
code += "        pos = lastAcceptPos;\n";
code += "        cout << lastAcceptId;\n";
code += "    } else {\n";
code += "        cout << \"Error\";\n";
code += "    }\n";
code += "}\n\n";

code += "int main() {\n";
code += "    cout << \"Enter string to analyze: \";\n";
code += "    cin.getline(buffer, 1024);\n";
```

4. 实验测试

4.1 测试数据设计

digit=[0-9]

letter=[A-Za-z]

ID_101=letter(letter|digit)*

4.2 运行结果效果图

正则表达式转换

正规式定义: 加载文件 开始转换 NFA 转换表 DFA 转换表 最小化 DFA 表 词法分析 源码查看

```
digit=[0-9]
letter=[A-Za-z]
ID_101=letter(letter|digit)*
```

	状态	ε	digit	letter
1	1 (S)			2
2	2	3		
3	3	4,10		
4	4	5,7		
5	5			6
6	6	9		
7	7		8	
8	8	9		
9	9	4,10		
10	10 (A)			

NFA 转换表

正则表达式转换

正规式定义: 加载文件 开始转换 NFA 转换表 DFA 转换表 最小化 DFA 表 词法分析 源码查看

```
digit=[0-9]
letter=[A-Za-z]
ID_101=letter(letter|digit)*
```

	状态	包含NFA状态	digit	letter
1	1 (Start)	{1}	-	{10,5,4,3,2,7}
2	2 (Token 0)	{10,5,4,3,2,7}	{8,9,10,5,4,7}	{9,10,5,4,6,7}
3	3 (Token 0)	{9,10,5,4,6,7}	{8,9,10,5,4,7}	{9,10,5,4,6,7}
4	4 (Token 0)	{8,9,10,5,4,7}	{8,9,10,5,4,7}	{9,10,5,4,6,7}

DFA 转换表

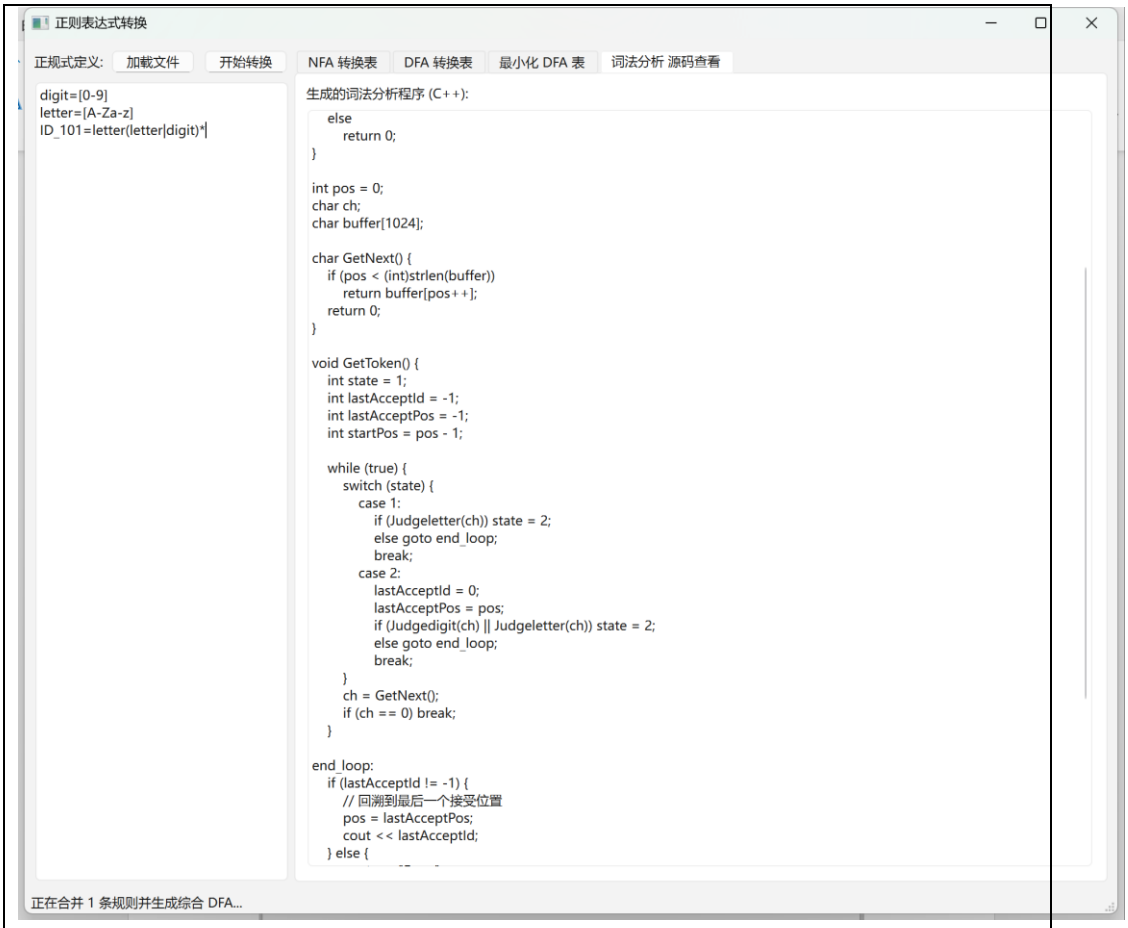
正则表达式转换

正规式定义: 加载文件 开始转换 NFA 转换表 DFA 转换表 最小化 DFA 表 词法分析 源码查看

```
digit=[0-9]
letter=[A-Za-z]
ID_101=letter(letter|digit)*
```

	状态	包含NFA状态	digit	letter
1	1 (Start)	{}	-	2
2	2 (Token 0)	{}	2	2

最小化 DFA 表



词法分析源码

四、实验总结（心得体会）

1. 深入理解自动机原理：通过手写 Thompson、子集构造、最小化算法，熟悉掌握正则→NFA→DFA 的完整流程，理解 ε 转移、闭包、状态等价性的核心思想。
2. 词法分析本质认知：词法分析本质是多模式串的最长匹配，基于 DFA 可高效实现，为后续语法分析打下基础。
3. 工程化实践提升：使用 Qt 实现图形界面，学会将算法逻辑与界面分离，提升代码模块化、可维护性。
4. 算法细节重要性：转义处理、别名展开、最长匹配、状态回溯等细节直接决定系统正确性，培养严谨的编码与调试能力。

五、参考文献：

正则表达式->NFA->DFA->最小化 DFA 图并生成词法分析程序：

https://blog.csdn.net/weixin_50549897/article/details/134065599